

Chapter 00 ~ 02 — 지금까지의 큰 흐름

- 0. 0~2 챕터를 정리하는 한 문장
전체 흐름 다이어그램
- 1. Chapter 00 — AI의 도구와 학습의 원리
 - 0-1. AI를 위한 Python (NumPy / 선형대수)
 - 0-2. 선형 회귀와 로지스틱 회귀
 - 1. 회귀(Regression)라는 단어
 - 2. 학습의 3단계 구조
 - 3. 경사 하강법 — 안개 낀 산 비유
 - 4. 회귀에서 분류로
- 2. Chapter 01 — 실무 파이프라인과 신경망의 깊이
 - 1-1. EDA와 머신러닝 파이프라인
 - 1. 표준 ML 파이프라인 (반드시 암기)
 - 2. 핵심 개념 정리
 - 3. scikit-learn 3단계 패턴
 - 4. 평가 지표
 - 1-2. MLP와 PyTorch
 - 1. 딥러닝 학습 루프 (모든 딥러닝의 공통 골격)
 - 2. 핵심 구성 요소
 - 3. Scikit-learn vs PyTorch
- 3. Chapter 02 — 언어를 다루는 AI와 LLM 활용
 - 2-1. 토큰화, 임베딩, 그리고 Transformer
 - ① 텍스트가 숫자가 되는 과정
 - 2. 시퀀스 모델의 발전사 — 각 단계는 앞 단계의 한계를 푸는 과정
 - 3. 아키텍처별 계보
 - 2-2. LLM 활용과 합성 데이터 생성
 - 1. 프롬프트의 기본 구조
 - 2. 프롬프팅 기법 비교
 - 3. 합성 데이터를 쓰는 이유
 - 4. 파이프라인
 - 5. 안전한 API 사용 원칙
- 4. 세 챕터를 관통하는 4가지 개념
- 5. 개념 점검 체크리스트
- 6. 앞으로 남은 과정

Chapter 00 ~ 02 — 지금까지의 큰 흐름

하나의 줄기를 살펴보기

0. 0~2 챕터를 정리하는 한 문장

" $y = wx + b$ 하나를 이해하고 => 그것을 층으로 쌓고 => 그것을 언어에 적용한다."

챕터	답하는 질문	한 줄 요약
00	기계는 대체 무엇을 "학습"하는가	숫자를 다루는 도구를 익히고, $y = wx + b$ 하나로 학습의 원리를 끝까지 파헤친다
01	그 원리를 어떻게 실무 파이프라인으로 만드는가	지저분한 데이터를 다듬고, 검증하고, 모델을 층으로 쌓는다
02	그 신경망을 어떻게 언어에 적용하는가	텍스트를 숫자로 바꾸고, Transformer를 거쳐 LLM 활용까지 간다

전체 흐름 다이어그램



1. Chapter 00 — AI의 도구와 학습의 원리

0-1. AI를 위한 Python (NumPy / 선형대수)

핵심 질문: AI는 왜 하필 벡터와 행렬로 계산하는가

개념	정의	직관적 비유
벡터(Vector)	원점 기준의 방향과 크기	지도 위의 "북동쪽으로 5블록" 화살표
행렬(Matrix)	벡터들의 모음	화살표 여러 개를 표로 묶은 것
스칼라 배	모든 원소에 같은 수를 곱함	사진을 확대·축소
선형 변환	공간을 늘리고 돌리되 원점은 유지	포토샵의 확대·회전·반전
NumPy 배열	숫자 계산에 특화된 배열	계산 전용으로 개조한 파이썬 리스트

- 행렬 곱셈 연산자 `@`, 배열 초기화, `shape`, 브로드캐스팅
- 자연상수 `e`와 미분의 기초 => 이후 경사 하강법의 사전 준비
- matplotlib 시각화, pandas DataFrame 기초

이 챕터가 존재하는 이유

데이터 1만 개를 for문으로 하나씩 계산하면 느리다. 행렬 연산으로 한 번에 처리하기 때문에 GPU 가속이 의미를 갖는다. 즉 선형대수는 "수학 과목"이 아니라 AI의 속도를 담당하는 실행 엔진이다.

0-2. 선형 회귀와 로지스틱 회귀

핵심 질문: 기계가 "학습한다"는 말은 정확히 무슨 뜻인가

1. 회귀(Regression)라는 단어

프랜시스 골턴의 어원에서 출발한다. 흠어진 데이터가 결국 하나의 선으로 돌아간다(regress)는 의미다.

2. 학습의 3단계 구조

1. 모델	$y = wx + b$	(w, b가 곧 "학습되는 것")
↓		
2. 손실 함수	MSE	(지금 모델이 얼마나 틀렸는지 = 모델의 성적표)
↓		
3. 최적화	경사 하강법	(성적표를 보고 w, b를 조금씩 고친다)

3. 경사 하강법 — 안개 낀 산 비유

비유	실제 용어	의미
산에서 내려가기	경사 하강법	손실이 낮아지는 방향으로 이동
한 걸음의 보폭	학습률(learning rate)	너무 크면 골짜기를 지나치고, 너무 작으면 영원히 못 내려감
산을 한 바퀴 도는 것	에포크(epoch)	전체 데이터를 한 번 다 훑은 것
한 번에 확인할 범위	배치(batch)	한 번의 업데이트에 사용하는 데이터 묶음
가짜 웅덩이	지역 최솟값	진짜 바닥이 아닌 곳에 갇히는 현상

4. 회귀에서 분류로

상황	사용 함수	출력
이진 분류 (합격/불합격)	시그모이드	0~1 사이 확률값 1개
다중 분류 (강아지/고양이/새)	소프트맥스	합이 1이 되는 확률 분포

방법	정규 방정식	경사 하강법
방식	수식으로 한 번에 정답 도출	반복하며 더듬어 찾아감
장점	정확, 반복 불필요	데이터가 커도 동작, 딥러닝에 적용 가능
한계	데이터·특성이 많으면 계산 폭발	학습률 설정 필요, 시간 소요

이것이 **GPT**의 최소 단위 부품이다

GPT의 "파라미터 수천억 개"라는 표현에서 그 파라미터의 정체가 바로 여기서 배운 **w**와 **b**다.

또한 LLM API의 `temperature` 파라미터는 **소프트맥스의 출력 분포를 얼마나 평평하게 만들 것인가**를 조절하는 값이다. 값이 낮으면 확률이 높은 토크에만 몰려 결정론적으로 답하고, 높으면 분포가 퍼져 다양한 답을 낸다. Chapter 02의 프롬프팅 실습에서 이 개념이 그대로 다시 등장한다.

2. Chapter 01 — 실무 파이프라인과 신경망의 깊이

1-1. EDA와 머신러닝 파이프라인

핵심 질문: 현실의 데이터는 왜 바로 모델에 넣을 수 없는가

Chapter 00에서는 깨끗한 데이터로 모델만 학습했다. 그러나 실무에서는 **모델링보다 데이터 정리에 더 많은 시간이 소요된다.**

1. 표준 ML 파이프라인 (반드시 암기)

데이터 수집·이해 (EDA)

↓

Train / Valid / Test 분할

↓

전처리 (결측치·이상치·스케일링)

↓

모델 학습

↓

평가 (지표 계산 + 교차검증)

↓

해석 및 개선

2. 핵심 개념 정리

개념	의미	주의점
EDA	분포·상관관계·결측치·이상치를 탐색	상관관계 ≠ 인과관계
결측치 / 이상치	빈 칸 / 동떨어진 값	IQR 기준으로 이상치 판별
Train/Valid/Test	학습 / 튜닝 / 최종 평가용 분리	Test는 학습에 절대 사용 금지
StandardScaler	평균 0, 표준편차 1로 표준화	fit 은 train 에만, test에는 transform만
과소적합	학습 데이터조차 못 맞춤	모델 복잡도를 올림
과대적합	학습만 잘하고 새 데이터에서 실패	규제, 데이터 증강, Early Stopping
교차검증	데이터를 여러 폴드로 나눠 반복 평가	일반화 성능을 안정적으로 추정

3. scikit-learn 3단계 패턴

```
1. 인스턴스 선언      model = LogisticRegression()
2. .fit()             학습 또는 통계량 계산
3. .predict() / .transform()  예측 또는 변환
```

4. 평가 지표

지표	의미	언제 쓰는가
Accuracy	전체 중 맞춘 비율	클래스가 균형일 때만 신뢰 가능
F1-Score	정밀도와 재현율의 조화 평균	불균형 데이터의 기본 지표
ROC-AUC	0과 1을 구분하는 능력	임계값과 무관한 성능 비교

- 비지도학습 맛보기: PCA(차원 축소)와 KMeans(군집화)로 레이블 없이 데이터 구조를 관찰

1-2. MLP와 PyTorch

핵심 질문: 선 하나로 안 되면 어떻게 하는가

선형 모델은 직선(평면)으로만 데이터를 나눈다. 곡선이 필요한 문제는 층을 쌓고 활성화 함수를 끼워 넣어 해결한다. 이것이 MLP(다층 퍼셉트론)이며 딥러닝의 출발점이다.

1. 딥러닝 학습 루프 (모든 딥러닝의 공통 골격)

```
Data -> DataLoader -> Model -> Loss -> Backward -> Optimizer
                                   ↓
                                   (파라미터 갱신 후 반복)
```

```
optimizer.zero_grad()      # 1. 이전 기울기 초기화
outputs = model(inputs)    # 2. 순전파
loss = criterion(outputs, labels) # 3. 손실 계산
loss.backward()            # 4. 역전파 (기울기 계산)
optimizer.step()           # 5. 파라미터 업데이트
```

이 5줄은 Chapter 03, 04까지 형태를 거의 바꾸지 않고 계속 등장한다.

2. 핵심 구성 요소

요소	역할	Chapter 00-01과의 연결
<code>torch.Tensor</code>	기본 자료구조	NumPy 배열 + GPU + 자동 미분
<code>Autograd</code>	연산을 추적해 기울기 자동 계산	0-2에서 손으로 계산한 미분을 자동화
<code>nn.Module</code>	모델 정의의 기본 클래스	<code>y = wx + b</code> 를 클래스로 감싼 것
<code>nn.Sequential</code>	레이어를 순서대로 묶는 컨테이너	층 쌓기
<code>CrossEntropyLoss</code>	다중 분류 손실 함수	내부에 소프트맥스 포함
<code>Optimizer (Adam)</code>	기울기로 파라미터 갱신	경사 하강법의 개량형
<code>DataLoader</code>	배치 생성·셔플링	0-2의 "배치" 개념을 코드로 구현
<code>train() / eval()</code>	학습·평가 모드 전환	Dropout 등의 동작이 달라짐
<code>Early Stopping</code>	검증 손실이 나빠지면 중단	과대적합 방지

3. Scikit-learn vs PyTorch

항목	Scikit-learn	PyTorch
학습	<code>model.fit()</code> 한 줄	for문으로 직접 제어
내부 동작	블랙박스	전 과정 제어 가능
GPU	✗	✓
용도	정형 데이터 · 전통 ML	딥러닝 전반

PyTorch가 어렵게 느껴지는 이유는 새로운 개념이 많아서가 아니라, **Chapter 00**에서 직접 손으로 하던 일을 코드가 대신하기 때문이다. `loss.backward()`는 0-2에서 종이에 계산한 편미분과 정확히 같은 값을 만든다. 실습에서 손계산 값과 `.grad` 값이 일치하는 것을 직접 확인했다.

3. Chapter 02 — 언어를 다루는 AI와 LLM 활용

2-1. 토큰화, 임베딩, 그리고 Transformer

핵심 질문: 신경망은 숫자만 먹는데, 텍스트는 어떻게 넣는가

① 텍스트가 숫자가 되는 과정

"영화 재밌어요"

↓ 토큰화 (Tokenization)

["영화", "재밌", "##어요"]

↓ 정수 ID 매핑

[452, 1893, 77]

↓ 임베딩 (nn.Embedding)

[[0.21, -1.03, ...], [...], [...]] # 각 토큰이 768차원 벡터로

↓

신경망 입력

- 토큰 ID 자체에는 의미가 없다. 의미는 임베딩 벡터의 위치가 담당한다.
- 의미가 비슷한 단어는 벡터 공간에서 가까이 놓이도록 학습된다.

2. 시퀀스 모델의 발전사 — 각 단계는 앞 단계의 한계를 푸는 과정

모델	핵심 아이디어	남은 한계
RNN	이전 시점의 은닉 상태를 다음 시점에 전달	문장이 길면 앞부분 정보 소실
LSTM	게이트로 기억할 것과 버릴 것을 구분	여전히 순차 처리 => 느림
Encoder-Decoder	입력을 문맥 벡터로 압축 후 디코딩	긴 문장을 벡터 하나로 압축하는 병목
Attention	매 시점마다 입력 전체에서 중요한 곳에 가중치 부여	—
Transformer	Self-Attention만으로 구성, 순차 처리 제거	계산량이 길이의 제곱에 비례

3. 아키텍처별 계보

구조	대표 모델	잘하는 일
Encoder-only	BERT	문장 이해 — 분류, 감성 분석, 검색
Decoder-only	GPT	문장 생성 — 챗봇, 요약, 코드 작성
Encoder-Decoder	T5, 번역 모델	입력을 받아 다른 형태로 변환 — 번역

- Huggingface `transformers` 로 사전학습 모델을 불러와 추론까지 수행

2-2. LLM 활용과 합성 데이터 생성

핵심 질문: 이미 만들어진 LLM을 어떻게 도구로 쓰는가

Chapter 02-1까지가 "LLM을 만드는 원리"였다면, 02-2는 "LLM을 쓰는 기술"이다.

1. 프롬프트의 기본 구조

- | | |
|--------------------|--------------------------|
| 1. 역할(Role) 설정 | "당신은 10년 경력의 영화 평론가입니다." |
| 2. 목표(Task) 명확화 | "요청 장르에 맞는 영화를 추천하세요." |
| 3. 조건(Constraints) | "JSON 형식, 추천 이유 포함" |
| 4. 출력 형식 지정 | ```json { ... } ``` |

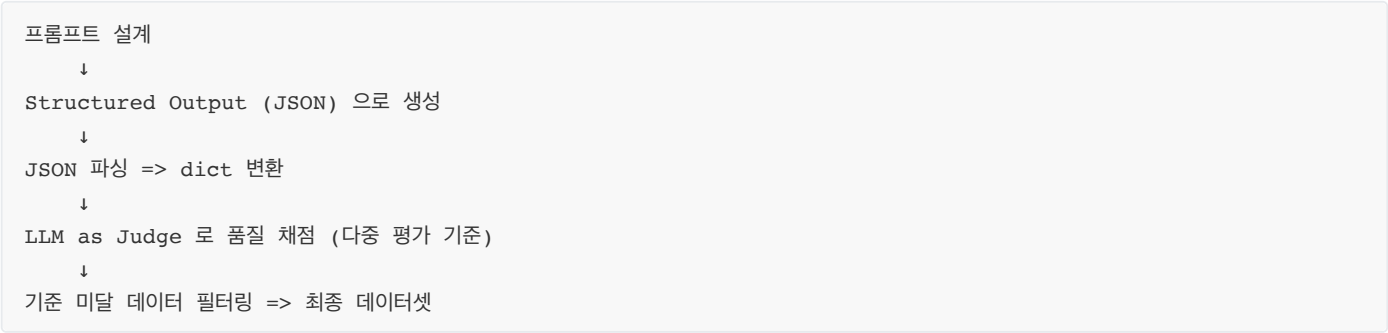
2. 프롬프팅 기법 비교

기법	방식	특징	적합한 상황
Zero-shot	예시 없이 지시만	가장 간단하나 출력 형식이 흔들림	단순 질의
Few-shot	예시 몇 개를 함께 제공	형식·톤이 일관됨	정해진 포맷의 대량 생성
CoT	단계별로 추론하게 유도	논리적이지만 토큰 소모 증가	복잡한 판단·계산

3. 합성 데이터를 쓰는 이유

- 1. 데이터 부족 해소 — 희귀 케이스·엣지 케이스 확보
- 2. 비용 절감 — 수집·라벨링 대비 저렴
- 3. 개인정보 보호 — 실제 개인 데이터 없이 유사 분포 생성
- 4. 테스트 다양화 — 드물게 발생하는 상황 재현

4. 파이프라인



5. 안전한 API 사용 원칙

원칙	이유
API 키는 환경변수(<code>os.environ</code> , <code>.env</code>) 로 관리	하드코딩 시 저장소 유출로 그대로 노출
모델명·예상 토큰·비용을 코드 상단 주석에 명시	실행 전 비용을 예측 가능하게 함
결정론적 설정(시드, 낮은 다양성 파라미터) 명시	실습 결과의 재현성 확보
AI가 생성한 결과물임을 명시적으로 고지(disclosure)	합성 데이터가 실제 데이터로 오인되는 것을 방지

**** LLM as Judge의 한계****
LLM 평가자는 저렴하고 빠르지만 완벽하지 않다. 자기가 생성한 답변에 후한 점수를 주는 편향, 긴 답변을 선호하는 편향이 알려져 있다. 현업에서는 LLM Judge를 1차 필터로 두고, 상위·하위 구간만 사람이 검수하는 **Human-in-the-loop** 구조로 운영한다. 전량을 LLM 판단에만 맡기지 않는다.

4. 세 챕터를 관통하는 4가지 개념

교육 과정 내내 이름만 바뀌 반복 등장하는 개념이다. 이 4가지가 연결되면 전체 흐름이 잡힌다.

개념	Ch 00	Ch 01	Ch 02
1. 파라미터 (w, b)	직접 손으로 계산	<code>nn.Linear</code> 의 weight/bias	Transformer의 수십억 개 가중치
2. 손실 함수	MSE, BCE	CrossEntropyLoss	사전학습 모델의 학습 목표
3. 경사 하강법	안개 낀 산 비유	Adam 옵티마이저	대규모 사전학습의 동력
4. 일반화 (과적합 방지)	Train/Test 분리	교차검증 / Early Stopping	사전학습 후 파인튜닝 전략

5. 개념 점검 체크리스트

Chapter 00

- ☐ 벡터와 행렬이 AI 계산에서 왜 필요한지 설명할 수 있다
- ☐ `y = wx + b`에서 학습되는 대상이 무엇인지 말할 수 있다
- ☐ MSE가 왜 "모델의 성적표"인지 설명할 수 있다
- ☐ 학습률·에포크·배치의 차이를 구분할 수 있다
- ☐ 시그모이드와 소프트맥스의 사용 상황을 구분할 수 있다

Chapter 01

- ☐ ML 파이프라인 6단계를 순서대로 말할 수 있다
- ☐ 스케일러의 `fit`을 train에만 적용하는 이유를 설명할 수 있다
- ☐ 정확도만 믿으면 안 되는 사례를 하나 들 수 있다
- ☐ 과소적합과 과대적합의 해결 방향이 반대임을 안다
- ☐ PyTorch 학습 루프 5단계를 순서대로 쓸 수 있다
- ☐ Autograd가 무엇을 자동화하는지 설명할 수 있다

Chapter 02

- ☐ 토큰 ID와 임베딩 벡터의 차이를 설명할 수 있다
- ☐ RNN에서 Transformer까지의 발전 이유를 한 줄씩 말할 수 있다
- ☐ BERT와 GPT의 구조적 차이와 용도를 구분할 수 있다
- ☐ Zero-shot / Few-shot / CoT를 상황에 맞게 선택할 수 있다
- ☐ API 키를 환경변수로 관리해야 하는 이유를 설명할 수 있다
- ☐ 합성 데이터의 품질을 검증하는 절차를 설명할 수 있다

6. 앞으로 남은 과정

지금까지는 정형 데이터(숫자, 표)와 텍스트를 다뤘다. 이후 과정은 다음 축으로 확장된다.

Ch 00~02	숫자와 텍스트	=> 학습의 원리, 신경망, LLM의 기초
↓		
Ch 03	이미지	=> CNN, 전이학습, 이미지 생성 및 평가
↓		
Ch 04	응용 시스템	=> RAG, AI Agent (LLM을 실제 서비스로 조립)

Chapter 00~02에서 만든 개념들이 사라지지 않고 그대로 재사용된다는 점이 중요하다.

예를 들어 Chapter 03의 전이학습에서도 **PyTorch 학습 루프 5단계**가 그대로 등장하고, Chapter 04의 RAG 검색에서도 **임베딩 벡터의 유사도**가 그대로 핵심 원리로 쓰인다.